MACHINE LEARNING

Module Code: CSC83101

LECTURE

TUYISHIMIRE Joseph

tuyishimirejo1992@gmail.com

UNIVERSITY Of KIGALI

Decision Tree in Machine Learning

A decision tree is a supervised learning algorithm used for both classification and regression tasks. It has a hierarchical tree structure which consists of a root node, branches, internal nodes and leaf nodes. It works like a flowchart help to make decisions step by step where:

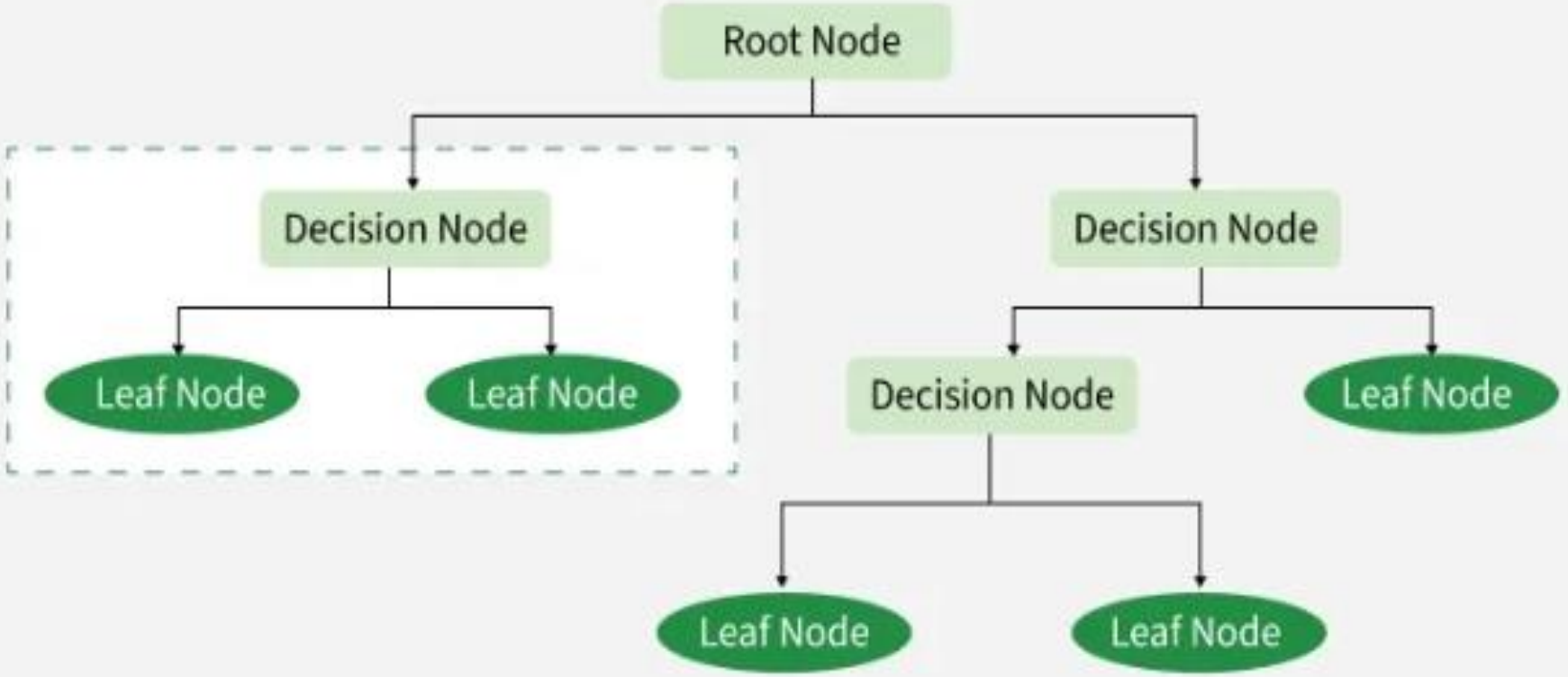
- Internal nodes represent attribute tests
- Branches represent attribute values
- Leaf nodes represent final decisions or predictions.

UNIVERSITY Of KIGALI

Decision trees are widely used due to their interpretability, flexibility and low preprocessing needs.

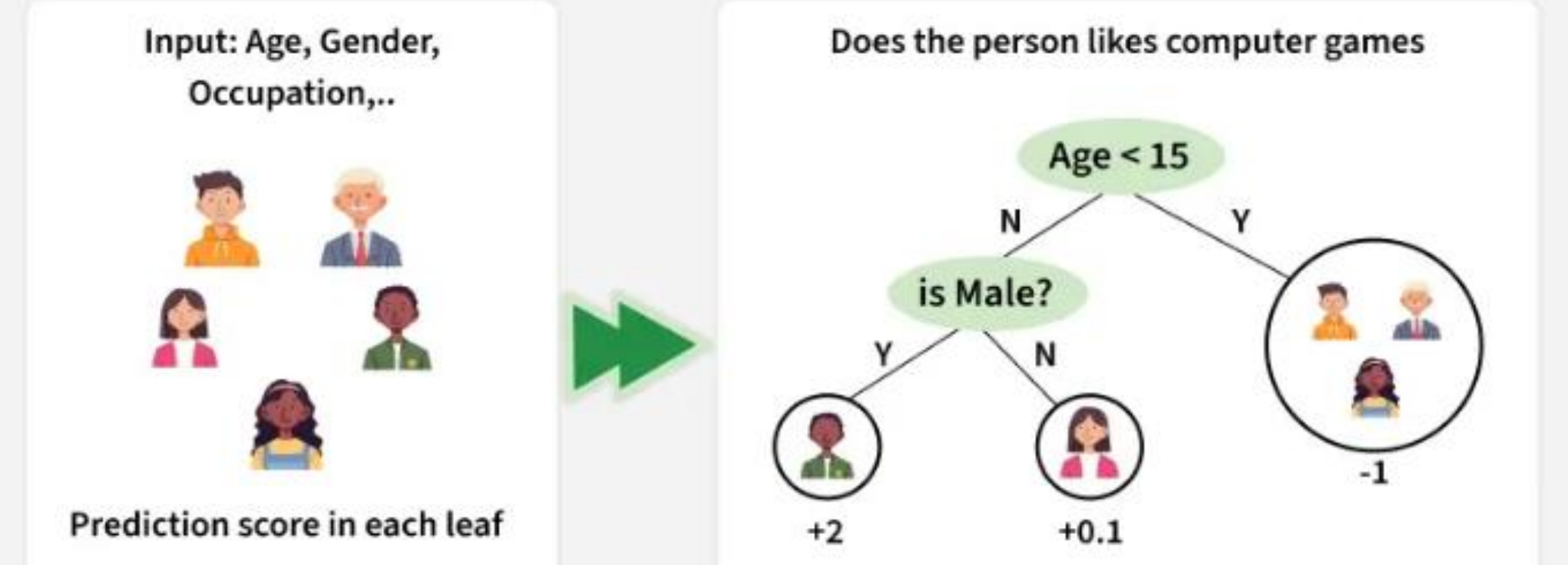
Decision Tree

A decision tree is a flowchart-like model used for classification and regression. It splits data based on features to make predictions . It looks like:



Working of Decision Tree

The model checks conditions like age and gender to split users into groups. Each group (leaf node) gets a prediction score based on user preferences for computer games.



Applications of Decision Trees

Finance	Optimizing loan approvals by evaluating multiple variables
Medicine	Diagnosing health conditions based on symptoms and medical history.
Machine Learning	Building blocks for algorithms handling classification and regression.
Education	Predicting student GPA from performance in core courses

UNIVERSITY of KIGALI

Splitting Criteria In Decision Tree

In decision trees, splitting criteria help decide which feature to split on at each node. The two most common criteria are:

Gini Index

$$I_G = 1 - \sum_{j=1}^{c} p_j^2$$

P_j : proportion of the samples that belongs to class c for a particular node

Entropy

$$I_H = -\sum_{j=1}^{c} p_j log_2(p_j)$$

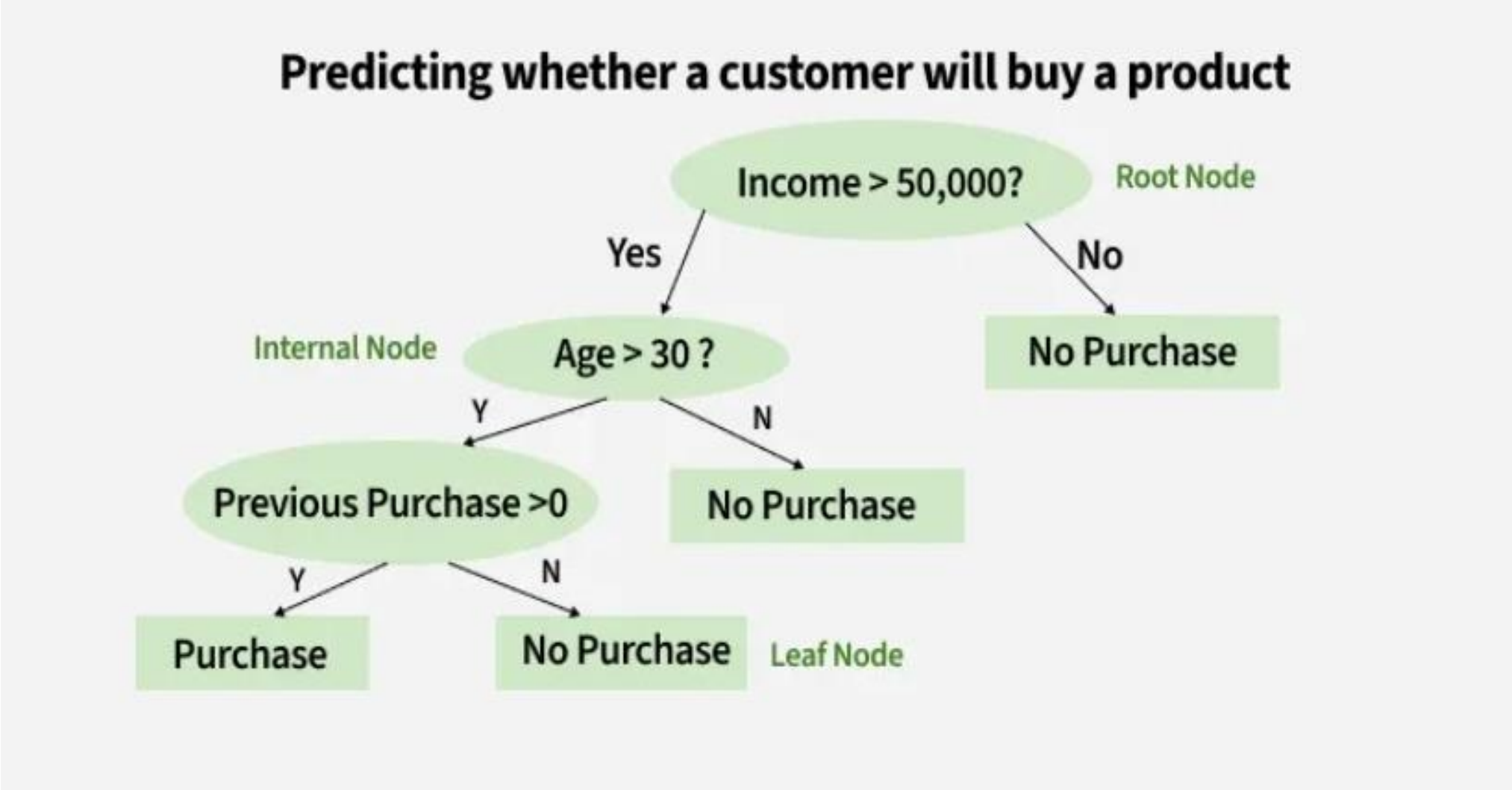
p_j: proportion of the samples that belongs to class c for a particular node.

*This is the the definition of entropy for all non-empty classes (p ne0) The entropy is 0 if all samples at a node belong to the same class.

UNIVERSITY Of KIGALI

How Does a Decision Tree Work

A decision tree splits the dataset based on feature values to create pure subsets ideally all items in a group belong to the same class. Each leaf node of the tree corresponds to a class label and the internal nodes are feature-based decision points. Let's understand this with an example


Predicting whether a customer will buy a product
Income > 50,000?
Root Node
Yes
No
Internal Node
Age > 30 ?
No Purchase
Y
N
Previous Purchase >0
No Purchase
Y
N
Purchase
No Purchase
Leaf Node

Let’s consider a decision tree for predicting whether a customer will buy a product based on age, income and previous purchases: Here's how the decision tree works:

1. Root Node (Income)

First Question: "Is the person's income greater than $50,000?"

- If Yes, proceed to the next question.
- If No, predict "No Purchase" (leaf node).

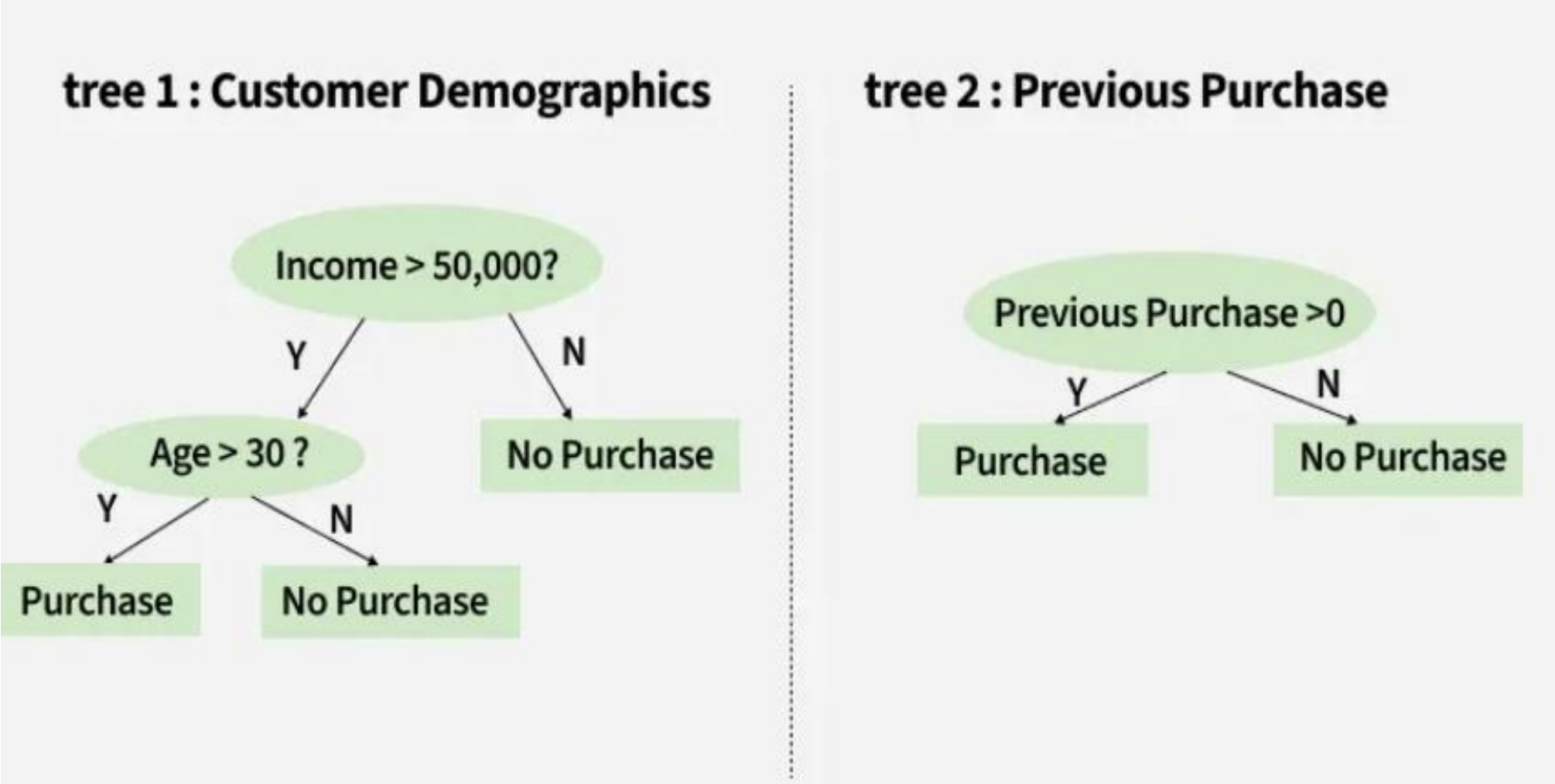
2. Internal Node (Age):

If the person's income is greater than $50,000, ask: **"Is the person's age above 30?"**

- If Yes, proceed to the next question.
- If No, predict "No Purchase" (leaf node).

3. Internal Node (Previous Purchases):

- If the person is above 30 and has made previous purchases, predict "Purchase" (leaf node).
- If the person is above 30 and has not made previous purchases, predict "No Purchase" (leaf node).


tree 1 : Customer Demographics
Income > 50,000?
Y
N
Age > 30 ?
No Purchase
Y
N
Purchase
No Purchase
tree 2 : Previous Purchase
Previous Purchase >0
Y
N
Purchase
No Purchase

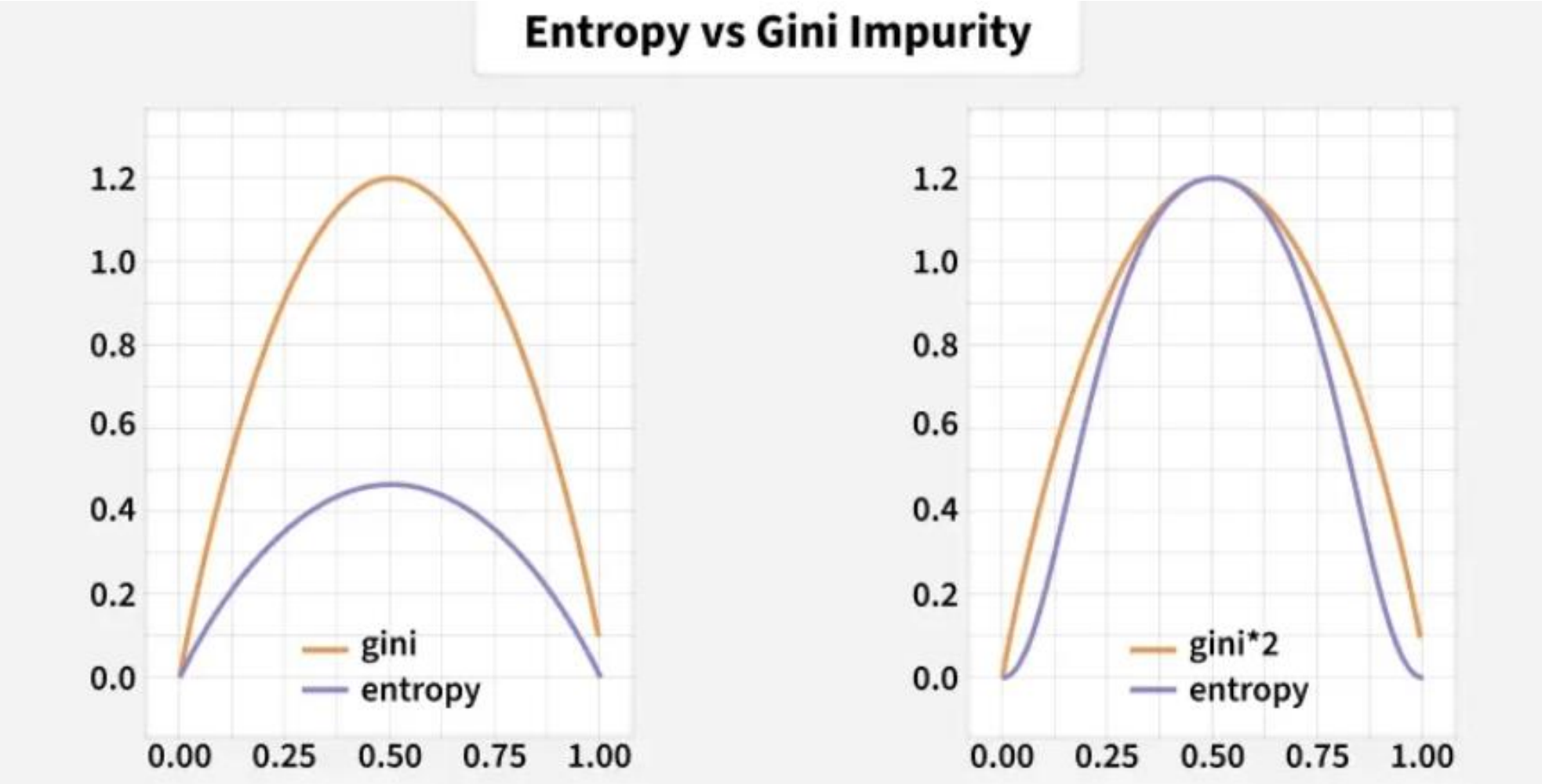
Splitting Criteria in Decision Trees

- In a Decision Tree, the process of splitting data at each node is important. The splitting criteria finds the best feature to split the data on.
- **Gini Impurity**: This criterion measures how "impure" a node is. The lower the Gini Impurity the better the feature splits the data into distinct categories.
- **Entropy**: This measures the amount of uncertainty or disorder in the data. The tree tries to reduce the entropy by splitting the data on features that provide the most information about the target variable.

UNIVERSITY Of KIGALI

Gini Impurity and Entropy in Decision Tree

Decision Trees are classification models that split data into nodes based on feature values. To determine the best split, they rely on impurity metrics that evaluate how mixed a node's class distribution is. Gini Impurity and Entropy are two measures used in decision trees to decide how to split data into branches. Both help determine how mixed or pure a dataset is, guiding the model toward splits that create cleaner groups.


Entropy vs Gini Impurity
1.2
1.0
0.8
0.6
0.4
0.2
0.0
0.00
0.25
0.50
0.75
1.00
gini
entropy
gini*2
entropy

Need for Impurity Measures

Some common reasons why impurity criteria are essential in decision tree learning are:

- Prevents random or uninformative splits that reduce predictive strength.
- Helps isolate class boundaries for better interpretability.
- Controls node quality and prevents over-growth of branches.
- Reduces classification ambiguity by separating noisy samples.
- Maintains model consistency across varied dataset

1. Gini Impurity

Gini Impurity checks how often a randomly selected sample would be mislabeled if assigned by class probability. It is computationally simple and used in tree-based classifiers

Formula:

$$\text{Gini} = 1 - \sum_{i=1}^{n} p_i^2$$

Where p_i is the probability of class i.

Properties:

- Lower values indicate cleaner and more homogeneous nodes.
- Nodes become pure when all samples belong to one class.
- Slightly biased toward dominant classes during split selection.

2. Entropy

Entropy measures uncertainty in a node's class distribution and originates from information theory. Higher entropy indicates greater disorder among class labels.

Formula:

$$\text{Entropy} = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

Where p_i represents the proportion of class i in the node.

Properties:

- Zero entropy corresponds to perfectly pure splits.
- Sensitive to small fluctuations across class ratios.
- Often yields balanced splits with meaningful boundaries

Applications

- **Fraud Detection:** Helps differentiate legitimate behaviors from suspicious anomalies, supporting real-time financial monitoring.
- **Customer Behavior Classification:** Enables marketing segmentation by grouping users with similar interaction patterns, improving personalized outreach.
- **Medical Predictions:** Assists in screening symptoms or diagnostic attributes, allowing healthcare systems to classify risk more accurately.
- **Quality Inspection:** Detects faulty products by separating normal sensor readings from defective patterns within manufacturing pipelines.
- **Document Categorization:** Organizes incoming text data into relevant topics, improving search and retrieval efficiency in large content systems.

Python | Decision tree implementation

A decision tree is a popular supervised machine learning algorithm used for both classification and regression tasks. It works with categorical as well as continuous output variables and is widely used due to its simplicity, interpretability and strong performance on structured data.

How Decision Trees Work

Import necessary libraries

- from sklearn.tree import DecisionTreeClassifier
- from sklearn.model_selection import train_test_split
- from sklearn.metrics import classification_report, confusion_matrix

Purpose: Import tools from scikit-learn to build and evaluate a Decision Tree classification model.

- **DecisionTreeClassifier**: The machine learning model.
- **train_test_split:** To split data into training and testing sets.
- **classification_report and confusion_matrix:** Metrics to evaluate model performance.

UNIVERSITY Of KIGALI

2. Prepare your dataset

X = features (input variables)

y = target (labels)

Purpose: Define the input features (X) and the output target (y).

- Example:

For a dataset of people, X could be columns like Age, Experience, Nationality (encoded), and y could be a binary label like whether they attended a comedy show (1 for Yes, 0 for No).

3. Split the dataset into training and testing sets

- X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

Purpose: Split the data to train the model on one portion (X_train, y_train) and test its performance on unseen data (X_test, y_test).

- test_size=0.3 means 30% of data is reserved for testing.
- random_state=42 ensures the split is reproducible.

Example: If you have 100 rows, 70 will be used for training and 30 for testing

UNIVERSITY Of KIGALI

Initialize the Decision Tree classifier

- dt_clf = DecisionTreeClassifier(random_state=42)

Purpose: Create an instance of the Decision Tree model.

- random_state controls randomness for reproducibility (e.g., how the tree splits when multiple choices exist).
- You can customize parameters here, like max depth (max_depth), minimum samples per leaf (min_samples_leaf), etc.

Example: Creating a blank Decision Tree ready to learn from data.

UNIVERSITY Of KIGALI

5. Train (fit) the model on the training data

- dt_clf.fit(X_train, y_train)

Purpose: The model learns patterns from the training data.

- It builds a tree structure by splitting data at different feature values to best separate the target classes.

Example: Teaching the model that people above a certain age and experience tend to attend the show.

6. Make predictions on the test data

- y_pred = dt_clf.predict(X_test)

Purpose: Use the trained model to predict labels for the unseen test data.

- This evaluates how well the model generalizes.

Example: Model predicts which test samples are likely to attend (1) or not (0).

7. Evaluate the model's performance

- print(confusion_matrix(y_test, y_pred))
- print(classification_report(y_test, y_pred))

Confusion Matrix:

- Shows true positives, true negatives, false positives, and false negatives, helping you understand where the model is right or wrong.

Example output for binary classification (1=attended, 0=not):

- [[TN, FP],
- [FN, TP]]

Classification Report:

Includes precision, recall, F1-score for each class, and overall accuracy.

- **Precision:** How many predicted positives were correct?
- **Recall:** How many actual positives were identified?
- **F1-score:** Harmonic mean of precision and reca

8. Plot the decision tree

```
plt.figure(figsize=(12,8))
plot_tree(dt_clf,
          feature_names=X.columns,  # Names of the features
          class_names=True, # Shows class labels, set to list if
custom
          filled=True,     # Colors nodes by class
          rounded=True,   # Rounded corners for nodes
          fontsize=10)
plt.title("Decision Tree Visualization")
plt.show()
```

Explanation of plotting step:

- plot_tree() draws the tree structure.
- feature_names adds readable feature labels on splits.
- class_names shows target class names (True will show numbers; you can replace with a list of strings).
- filled=True colors nodes by majority class.
- rounded=True makes the boxes rounded and visually cleaner.
- plt.figure(figsize=(width, height)) adjusts the plot size for readability.

Example 1

You are working with data collected from a person who tracks whether they attend comedy shows based on information about the comedians performing. The dataset includes features related to the comedians such as:

- Age: The comedian's age.
- Experience: Years of experience the comedian has.
- Rank: A rating or rank assigned to the comedian.
- Nationality: The comedian's nationality (UK, USA, or N).
- Go: Whether the person decided to attend that comedian's show (YES or NO).
- The goal is to predict whether the person will attend a comedy show (Go) based on the attributes of the comedian.

UNIVERSITY of KIGALI

Problem Identification:

Type: This is a classification problem where the target variable (Go) is categorical with two classes: YES (attended) or NO (did not attend).

Goal: Build a predictive model that classifies if the person will attend a comedy show given the comedian's features.

Data Characteristics:

- Mixed data types (numerical: Age, Experience, Rank; categorical: Nationality).
- Small dataset size but potentially enough to identify patterns.
- Target is binary classification.

UNIVERSITY Of KIGALI

Step 1: Generate the dataset

```python
import pandas as pd

# Create the dataset
data = {
    'Age': [36, 42, 23, 52, 43, 44, 66, 35, 52, 35, 24, 18, 45],
    'Experience': [10, 12, 4, 4, 21, 14, 3, 14, 13, 5, 3, 3, 9],
    'Rank': [9, 4, 6, 4, 8, 5, 7, 9, 7, 9, 5, 7, 9],
    'Nationality': ['UK', 'USA', 'N', 'USA', 'USA', 'UK', 'N', 'UK', 'N', 'N', 'USA', 'UK', 'UK'],
    'Go': ['NO', 'NO', 'NO', 'NO', 'YES', 'NO', 'YES', 'YES', 'YES', 'YES', 'NO', 'YES', 'YES']
}

df = pd.DataFrame(data)
print(df)
```

Step 2: Preprocess the data

```python
"""Step 2: Preprocess the data
Encode categorical variables (Nationality and target Go)
Define features and target"""
from sklearn.preprocessing import LabelEncoder

# Encode 'Nationality'
le_nationality = LabelEncoder()
df['Nationality_encoded'] = le_nationality.fit_transform(df['Nationality'])

# Encode target 'Go' (YES=1, NO=0)
le_go = LabelEncoder()
df['Go_encoded'] = le_go.fit_transform(df['Go'])

# Features and target
X = df[['Age', 'Experience', 'Rank', 'Nationality_encoded']]
y = df['Go_encoded']
```

Step 3: Split dataset into training and test sets

```
#Step 3: Split dataset into training and test sets
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Step 4: Train the Decision Tree classifier

```
#Step 4: Train the Decision Tree classifier
from sklearn.tree import DecisionTreeClassifier

dt_clf = DecisionTreeClassifier(random_state=42)
dt_clf.fit(X_train, y_train)
```

Step 5: Evaluate the model

```
from sklearn.metrics import classification_report, confusion_matrix

y_pred = dt_clf.predict(X_test)

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Step 6: Visualize the Decision Tree

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(14,8))
plot_tree(dt_clf,
          feature_names=['Age', 'Experience', 'Rank',
'Nationality_encoded'],
          class_names=['NO', 'YES'],
          filled=True, rounded=True, fontsize=12)
plt.title("Decision Tree for Comedy Show Attendance")
plt.show()
```

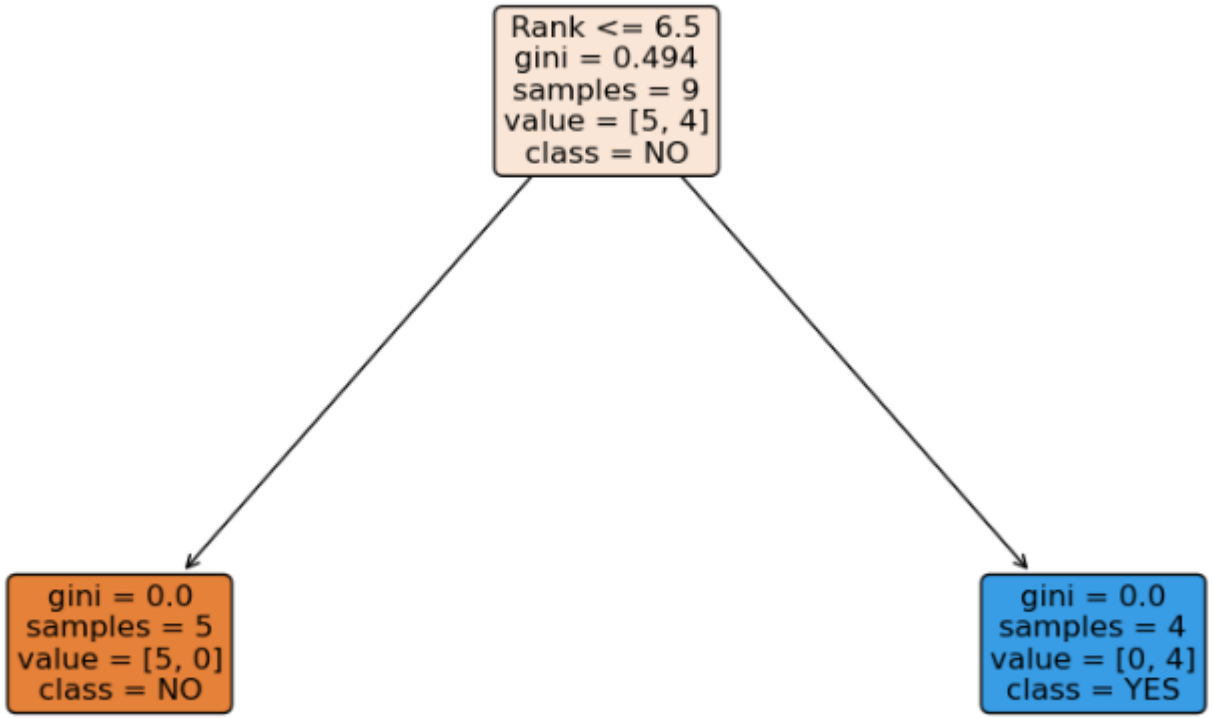
•**Precision for class 0 (NO):** 0.00
This means that none of the predictions made for "NO" were actually correct (no true negatives).
•**Recall for class 0:** 0.00
The model failed to identify any actual "NO" cases correctly.
•**F1-score for class 0:** 0.00
Reflects poor performance in detecting the "NO" class.
•**Precision for class 1 (YES):** 0.75
Of all instances predicted as "YES," 75% were correct.
•**Recall for class 1:** 1.00
The model correctly identified all actual "YES" cases (no false negatives).
•**F1-score for class 1:** 0.86
Indicates good balance between precision and recall for the "YES" class

```
Confusion Matrix:
 [[0 1]
 [0 3]]

Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00         1
           1       0.75      1.00      0.86         3

    accuracy                           0.75         4
   macro avg       0.38      0.50      0.43         4
weighted avg       0.56      0.75      0.64         4
```


Rank <= 6.5
gini = 0.494
samples = 9
value = [5, 4]
class = NO
gini = 0.0
samples = 5
value = [5, 0]
class = NO
gini = 0.0
samples = 4
value = [0, 4]
class = YES

1. The Root Node (Top)

This is the starting point of the decision-making process.

- **Split Criterion (** `Rank <= 6.5` **):** The model is checking if the "Rank" of a sample is 6.5 or lower. This is the feature used to divide the data.
- **Gini Impurity (** `gini = 0.494` **):** This measures the "impurity" or randomness of the node. A value close to 0.5 (in a binary split) means the data is almost evenly mixed between classes (5 "NO" and 4 "YES").
- **Samples (** `samples = 9` **):** There are 9 total data points being analyzed at this stage.
- **Value (** `value = [5, 4]` **):** Shows the distribution of classes. There are 5 samples of the first class and 4 of the second.
- **Class (** `class = NO` **):** If the tree stopped here, it would predict "NO" because that is the majority class.

These are the final outcomes of the model. In this specific graph, the tree has reached **maximum purity**.

- **Left Leaf (Orange):**
  - This represents samples where **Rank is $\leq$ 6.5**.
  - **Gini = 0.0:** This is a "pure" node, meaning all samples here belong to the same category.
  - **Result:** All 5 samples are "NO".
- **Right Leaf (Blue):**
  - This represents samples where **Rank is > 6.5**.
  - **Gini = 0.0:** Also a pure node.
  - **Result:** All 4 samples are "YES".

Key Observations

- **Discriminatory Power:** The feature "Rank" is a very strong predictor for this dataset. A single split was enough to perfectly separate the "YES" and "NO" classes.
- **Perfect Classification:** Because the Gini impurity in both bottom leaves is 0.0, the model has 100% accuracy on this specific training set.
- **Visual Coding:** The colors are meaningful. **Orange** usually represents one class, while **Blue** represents the other. The intensity of the color indicates how "pure" the node is (since these are solid colors, they are perfectly pure).

Would you like me to explain how the **Gini Impurity** formula is actually calculated, or perhaps help you write the code to generate a tree like this?